



INTERNET CONNECTIVITY FOR PRIVATE INSTANCES (NAT INSTANCE AND BASTION HOST)

Networking and **troubleshooting** go hand in hand, and let us see how we can **troubleshoot one**.

We have an instance that lives in a **private subnet** and cannot connect to the public internet because it contains important, sensitive data.

We want this instance to **download dependencies from the internet**, but we don't want the internet to talk to this EC2 instance. To make this possible, we will create a **NAT INSTANCE**.

A **NAT INSTANCE** is a **server** you create that allows **servers or EC2 instances** in **private subnets inside your network** to access the **internet** without exposing them to the **public internet or allowing the internet to talk to them**. We can think of **NAT instances** as the **middleman** for **internet traffic** for resources in a **private subnet**.

In **modern infrastructure setups**, people often use a **NAT GATEWAY** when designing their VPC, which is a managed AWS service, and is **more scalable** but **more expensive**.

Our **private instance** will be created in a **private subnet**, and the **NAT instance** will be created in a **public subnet**.

1) CREATE A SECURITY GROUP FOR THE nat_instance

I) **LOG IN** TO YOUR ACCOUNT, **CHOOSE** THE **eu-north-1** REGION, AND **SEARCH FOR EC2**.

II) IN YOUR **EC2 INSTANCE DASHBOARD**, LOCATE **NETWORK AND SECURITY**, CLICK ON **SECURITY GROUPS**, AND CLICK ON **CREATE SECURITY GROUP**.

III) IN THE **SECURITY GROUP** PAGE, GIVE YOUR **SECURITY GROUP** THE NAME **NAT_SG**, AND UNDER **VPC**, SELECT THE **DEFAULT VPC ID**.

IV) UNDER **INBOUND RULES**, ADD THE FOLLOWING RULES:

UNDER **TYPE**, CHOOSE **HTTP**, AND UNDER **SOURCE**, COPY AND PASTE THE DEFAULT VPC **PRIVATE IPV4 SUBNET CIDR BLOCK** WHERE YOU INTEND TO CREATE YOUR **PRIVATE EC2 INSTANCE**. I INTEND TO USE THE **RDS-Pvt-subnet-1** SUBNET FOR **MY PRIVATE INSTANCE**. THE **PRIVATE SUBNET CIDR BLOCK** OF THE **RDS-Pvt-subnet-1** IS **173.31.48.0/25**

UNDER **TYPE**, CHOOSE **HTTPS**, AND UNDER **SOURCE**, COPY AND PASTE THE DEFAULT VPC **PRIVATE IPV4 SUBNET CIDR BLOCK** WHERE YOU INTEND TO CREATE YOUR **PRIVATE EC2 INSTANCE**. I INTEND TO USE THE **RDS-Pvt-subnet-1** SUBNET FOR **MY PRIVATE INSTANCE**. THE **PRIVATE SUBNET CIDR BLOCK** OF THE **RDS-Pvt-subnet-1** IS **173.31.48.0/25**

UNDER **TYPE**, CHOOSE **SSH**, AND UNDER **SOURCE**, SELECT **CUSTOM**, AND CHOOSE **0.0.0.0/0**

V) UNDER **OUTBOUND RULES**, ADD THE FOLLOWING RULES:

UNDER **TYPE**, CHOOSE **HTTP**, AND UNDER **SOURCE**, SELECT **CUSTOM**, AND CHOOSE **0.0.0.0/0**

UNDER **TYPE**, CHOOSE **HTTPS**, AND UNDER **SOURCE**, SELECT **CUSTOM**, AND CHOOSE **0.0.0.0/0**

LEAVE THE OTHER DEFAULTS, AND CLICK **CREATE SECURITY GROUP**.

2) CREATE A NAT INSTANCE IN A PUBLIC SUBNET.

I) **LOG IN** TO YOUR ACCOUNT, **CHOOSE** THE **eu-north-1** REGION, AND **SEARCH FOR EC2**.

II) UNDER **NAME**, GIVE YOUR INSTANCE ANY NAME. FOR ME, I WILL GIVE MY INSTANCE THE NAME **nat_instance**

III) CHOOSE AN **AMAZON LINUX 2023 KERNEL AMI**. UNDER **INSTANCE TYPE**, CHOOSE A **t3 micro**.

III) UNDER **KEY PAIR**, CREATE A **KEY PAIR** FOR **SSH**. I WILL GIVE MY KEY PAIR THE NAME **nat_keypair**

IV) UNDER **NETWORK SETTINGS**, SELECT YOUR **DEFAULT VPC**.

V) ATTACH THE NAT INSTANCE **SECURITY GROUP** CALLED **NAT_SG**.

VI) LEAVE THE OTHER DEFAULTS, AND CLICK ON **LAUNCH INSTANCE**.

VI) ALLOW YOUR EC2 INSTANCE TO INITIALIZE BY LOOKING AT THE **STATUS CHECK** TO SEE **2/2** OR **3/3**.

VII) OPEN **CLOUDSHELL** AND UPLOAD YOUR DOWNLOADED PRIVATE **nat_keypair.pem** KEY PAIR.

VIII) **REMOTELY SSH** INTO YOUR EC2 INSTANCE BY CLICKING ON YOUR **nat_instance** EC2 INSTANCE, CLICK ON THE **CONNECT** BUTTON, CLICK ON THE **SSH CLIENT TAB**, FOLLOW ALL THE INSTRUCTIONS, AND CONNECT REMOTELY TO YOUR INSTANCE.

IX) **ENABLE IPTABLES.**

iptables acts as the **traffic controller** and **ID-changer** for EC2 instances in the **private subnets**. While the **NAT instance** itself is just a server, **iptables** is the **software rulebook** inside that server that tells it how to handle **packets passing through it**.

If a server in your **private subnet** wants to **download** an update from the internet, because it has a **private IP**, the **internet doesn't know how to "talk back"** to it.

The **private server** sends the request to the **NAT instance**, the **iptables** intercepts the packet, strips off the **private IP address**, and puts its own **Public IP** on the "From" label, and it remembers who asked for the data so that when the update comes back, it can hand it to the right private server.

INSTALL, ENABLE, AND START YOUR INSTALLED IPTABLES IN THE nat_instance EC2 INSTANCE BY TYPING THE COMMANDS BELOW.

```
sudo yum install iptables-services -y
```

```
sudo systemctl enable iptables
```

```
sudo systemctl status iptables
```

```
sudo systemctl start iptables
```

```
sudo systemctl status iptables
```

X) **ENABLE IP FORWARDING** FROM YOUR INSTALLED **IPTABLES** TO **PERSIST** AFTER YOUR **EC2 INSTANCE REBOOTS**. TYPE THE COMMANDS BELOW.

```
sudo nano /etc/sysctl.d/custom-ip-forwarding.conf
```

INSIDE THE CONFIGURATION FILE, ADD THE LINE

```
net.ipv4.ip_forward=1
```

TYPE ctrl + O

ENTER

ctrl + X

XI) TYPE THE COMMAND BELOW TO APPLY THE **CONFIGURATION FILE**.

```
sudo sysctl -p /etc/sysctl.d/custom-ip-forwarding.conf
```

XII) TYPE THE COMMAND BELOW TO SEE THE NAME OF THE **PRIMARY NETWORK INTERFACE** OF YOUR EC2 INSTANCE.

```
netstat -i
```

FROM THE RESULT BELOW, MY PRIMARY NETWORK INTERFACE IS ens5.

Iface	MTU	RX-OK	RX-ERR	RX-DRP	RX-OVR	TX-OK	TX-ERR	TX-DRP
ens5	9001	14036	0	0 0		2116	0	0
lo	65536	12	0	0 0		12	0	0

XIV) **CONFIGURE NAT** ON THE **nat_instance** EC2 INSTANCE.

IF YOUR PRIMARY NETWORK INTERFACE IS NOT ens5, REPLACE ens5 WITH THE PRIMARY NETWORK INTERFACE THAT YOU NOTED IN THE PREVIOUS STEP.

TYPE THE COMMANDS BELOW.

```
sudo /sbin/iptables -t nat -A POSTROUTING -o ens5 -j MASQUERADE
```

```
sudo /sbin/iptables -F FORWARD
```

```
sudo service iptables save
```

3) DISABLE SOURCE/DESTINATION CHECKS

Each EC2 instance performs source/destination checks by default. This means that the instance must be the **source or destination of any traffic it sends or receives**.

However, a **NAT instance** must be able to **send** and **receive traffic** when the **source** or **destination** is not itself, meaning that **we want traffic** to flow through it **to another service**, which is a **private instance** we will create as we progress.

Therefore, you must **disable source/destination** checks on the **NAT instance** to allow traffic flow through it.

I) LOCATE YOUR **nat_instance**, CLICK ON IT, AND LOCATE **ACTIONS**.

II) CLICK ON THE ACTIONS DROP-DOWN, LOCATE **NETWORKING**, CLICK ON IT, AND CLICK ON **CHANGE SOURCE/DESTINATION CHECK**.

III) IN THE **CHANGE SOURCE / DESTINATION CHECK** PAGE, CLICK ON THE **STOP** BOX AND CLICK ON **SAVE**.

4) UPDATE THE ROUTE TABLE IN YOUR DEFAULT VPC.

The **route table** for the **private subnet** that will host the **private instance** must have a route that **sends internet traffic** to the **NAT instance**.

I) SEARCH FOR VPC, LOCATE **ROUTE TABLES**, AND SELECT THE **RDS-Pvt-rt** ROUTE TABLE ATTACHED TO THE **DEFAULT VPC**.

THIS TABLE IS WHAT ROUTES PRIVATE RESOURCES WITHIN THE DEFAULT VPC LOCALLY. MAKE SURE THAT IT IS ATTACHED TO THE DEFAULT VPC WHERE YOU HAVE YOUR CREATED RESOURCES. ENSURE THIS BY CHECKING THAT THE DEFAULT VPC ID IS ATTACHED TO THIS ROUTE TABLE.

II) CLICK ON THE ROUTES TAB, CLICK ON EDIT ROUTES, ADD A ROUTE THAT FORWARDS ALL INTERNET TRAFFIC TO THE NAT INSTANCE.

CLICK ON ADD ROUTE, UNDER DESTINATION SEARCH FOR 0.0.0.0/0, UNDER TARGET, SELECT INSTANCE, AND ATTACH THE INSTANCE ID OF THE nat_instance (i-0e66c97394c1e5ca0) YOU CREATED FROM STEP 1. CLICK ON SAVE CHANGES.

ENSURE THAT THIS ROUTE TABLE IS ATTACHED TO ALL THE PRIVATE SUBNETS WITHIN THE DEFAULT VPC.

THE ABOVE CONFIGURATIONS ALLOW THE PRIVATE INSTANCE TO REACH THE INTERNET THROUGH THE NAT INSTANCE

If your private instance does:

curl https://google.com

Here's what happens:

Private EC2

↓ (route table says: 0.0.0.0/0 → NAT instance)

NAT Instance

↓

Internet Gateway

↓

Google

Then the response comes back:

Google → NAT Instance → Private EC2

5) UPDATE YOUR NAT INSTANCE SECURITY GROUP (NAT_SG)

To allow instances in your private subnet to send internet traffic to the NAT instance, add a rule to allow inbound and outbound ICMP traffic.

ICMP traffic is a type of **network message** computers use to check if **another device is reachable** and to report **network problems**.

ICMP stands for **Internet Control Message Protocol**.

You can think of it like this:

- Your computer sends a small “Are you there?” message.
- Another computer replies with “Yes, I’m here.”

That is **ICMP traffic**, and the **most common** example is the **ping command**, which we have used **throughout** this **Linux and Cloud Network Troubleshooting Course**.

For example, if you **ping google.com**, your computer sends **ICMP** requests to **Google’s server**, and Google replies with **ICMP responses**.

ICMP is mainly used for **testing connectivity between devices**, **checking if a server is online**, **diagnosing network issues**, and **reporting errors in the network**. In cloud platforms like **AWS**, security groups may **allow** or **block** **ICMP** traffic. Allowing **ICMP** lets you ping an **EC2 instance** to **test connectivity**.

For example, if **ICMP** is blocked, **ping** will fail even if the server is running, and if **ICMP** is allowed, ping can confirm the **instance is reachable**.

ADD A BASTION SERVER/HOST

To allow the **NAT instance** to serve as a **bastion server or host** for your **private instance**, add a rule to your **NAT security group (NAT_SG)** to allow **outbound SSH traffic** to the **private subnet or subnets** that will host your **private EC2 instances**.

A **bastion host** allows you to **remotely access a private instance** without exposing that **private instance** directly to the **internet**, mainly because **EC2 instances** have **no public IP**, and a **bastion host** makes it possible.

I) SEARCH FOR EC2, AND IN YOUR **EC2 INSTANCE DASHBOARD**, LOCATE **NETWORK AND SECURITY**, CLICK ON **SECURITY GROUPS**, SEARCH FOR THE **NAT_SG** SECURITY GROUP, AND CLICK ON IT.

II) ON THE **INBOUND RULES** TAB, CLICK ON **EDIT INBOUND RULES**. CLICK ON **ADD RULE**.

UNDER **TYPE**, CHOOSE **custom ICMP – IPV4**, UNDER **SOURCE**, CHOOSE **CUSTOM**, AND ENTER THE **PRIVATE IPV4 ADDRESS** OF THE **PRIVATE SUBNET** YOU ARE GOING TO CREATE YOUR PRIVATE EC2 INSTANCE IN.

YOU CAN ALSO ADD ALL THE **INDIVIDUAL PRIVATE IPV4 ADDRESSES** OF ALL THE **PRIVATE SUBNETS** IN YOUR **DEFAULT VPC** BY **DUPLICATING** THE **ABOVE PROCESS FOR ALL**.

THIS ALLOWS YOU TO PUT YOUR **PRIVATE EC2 INSTANCE** IN ANY **PRIVATE SUBNET**.

CLICK ON **SAVE RULES**.

III) ON THE **OUTBOUND RULES** TAB, CLICK ON **EDIT INBOUND RULES**. CLICK ON **ADD RULE**.

UNDER **TYPE**, CHOOSE **SSH**, UNDER **DESTINATION**, CHOOSE **CUSTOM**, AND ENTER THE **PRIVATE IPV4 ADDRESS** OF THE **PRIVATE SUBNET** YOU ARE GOING TO CREATE YOUR PRIVATE EC2 INSTANCE IN.

YOU CAN ALSO ADD ALL THE **INDIVIDUAL PRIVATE IPV4 ADDRESSES** OF ALL THE **PRIVATE SUBNETS** IN YOUR **DEFAULT VPC** BY **DUPLICATING** THE **ABOVE PROCESS FOR ALL**.

CLICK ON **ADD RULE**, UNDER **TYPE**, CHOOSE **custom ICMP – IPV4**, UNDER **DESTINATION**, CHOOSE **Anywhere-IPv4**.

CLICK ON **SAVE RULES**.

6) CREATE A PRIVATE INSTANCE IN A PRIVATE SUBNET

Launch a **private instance** into a **private subnet**. You must allow **SSH access from the NAT instance**, and you must use **the same key pair (nat_keypair)** you used for the **NAT instance**.

I) **SEARCH FOR EC2**.

II) UNDER **NAME**, GIVE YOUR INSTANCE ANY NAME. FOR ME, I WILL GIVE MY INSTANCE THE NAME **private_instance**

III) CHOOSE AN **AMAZON LINUX 2023 KERNEL AMI**. UNDER **INSTANCE TYPE**, CHOOSE A **t3 micro**.

IV) UNDER **KEY PAIR**, SELECT THE **nat_keypair**

V) UNDER **NETWORK SETTINGS**, SELECT YOUR **DEFAULT VPC**. UNDER **SUBNET**, SELECT A PRIVATE SUBNET. I WILL USE **RDS-Pvt-subnet-1** SUBNET. **AUTO ASSIGN PUBLIC IP IS AUTOMATICALLY DISABLED** WHEN YOU CHOOSE A **PRIVATE SUBNET**.

UNDER **FIREWALL (SECURITY GROUPS)**, GIVE YOUR **SECURITY GROUP** THE NAME **PRIVATE_SG**.

FOR **INBOUND RULE**, UNDER **TYPE**, SELECT **SSH**, UNDER **SOURCE TYPE**, SELECT **CUSTOM**, UNDER **SOURCE**, PASTE THE **IPV4 ADDRESS OF YOUR PUBLIC SUBNET** THAT HOSTS THE **NAT INSTANCE (172.31.32.0/20)**.

YOU CAN **DUPLICATE SSH ACCESS** FOR ALL YOUR **PUBLIC SUBNETS** LIKE THE ABOVE.

VI) LEAVE THE **OTHER DEFAULTS**, AND CLICK ON **LAUNCH INSTANCE**.

VII) LOCATE **NETWORK AND SECURITY**, CLICK ON **SECURITY GROUPS**, SEARCH FOR THE **PRIVATE_SG** SECURITY GROUP, AND CLICK ON IT.

CLICK ON THE **OUTBOUND RULES** TAB, CLICK ON **EDIT OUTBOUND RULES**.

CLICK ON **ADD RULE**, UNDER **TYPE**, CHOOSE **custom ICMP – IPV4**, UNDER **DESTINATION**, CHOOSE **Anywhere-IPv4**.

CLICK ON **SAVE RULES**.

7) TEST YOUR PRIVATE INSTANCE FROM YOUR NAT INSTANCE

You are going to verify that the **PRIVATE INSTANCE** in the **private subnet** can use your **NAT INSTANCE** in the **public subnet** to **communicate with the internet**.

You are going to **CONFIGURE SSH AGENT FORWARDING** to **enforce security standards** for our infrastructure.

SSH Agent Forwarding lets you use your local **SSH private keys (nat_keypair)** you downloaded at **EC2 instance creation** on the **EC2 instances (public or private)** you created without **copying** those **private keys (nat_keypair)** to the EC2 instance.

1. When you **SSH** from your **laptop** to a **bastion host (nat_instance)**
2. From the **bastion host (nat_instance)**, you need to **SSH** to the **private EC2 instance (private_instance)**.
3. **Without forwarding:** You will need to copy your **private key** to the **bastion host (nat_instance)**, which is a **high security risk** especially in this **periods of cyber threats**.
4. **With forwarding:** Your laptop **temporarily "forwards"** the **key** through the **connection**.
You can enable this by following the processes below.

I) **OPEN CLOUDSHELL** IN AWS OR YOUR **LOCAL MACHINE** IF YOU HAVE **AWS CONFIGURE** ENABLED. MAKE SURE YOU HAVE YOUR **PRIVATE KEY PAIR** UPLOADED IN THE **HOME DIRECTORY** WHERE YOU CHOOSE TO PERFORM YOUR TASK.

ENABLE THE **SSH AGENT** AND **SSH INTO THE NAT INSTANCE**. TYPE THE COMMANDS BELOW.

```
eval $(ssh-agent)                                # start the ssh agent
```

```
eval $(ssh-agent -k)                             # use when you want to kill the ssh agent
```

The SSH Agent is a program that holds your **SSH private keys in memory** so you don't have to keep entering your **passphrase**. It also lets you use your **local keys**, like your **private key pairs** on your **EC2 instance** or other **remote servers**, without copying them there.

ssh-add key.pem **# replace key.pem with your own keypair (nat_keypair.pem)**

ssh-add nat_keypair.pem

ssh -A ec2-user@nat.instance.public.ip.address

ssh -A [ec2-user@16.171.18.45](#)

The -A flag enables forwarding.

replace ec2-user@nat-instance-public-ip-address with your actual nat instance public IP, like [ec2-user@16.171.18.45](#)

II) CONFIRM THAT YOUR NAT INSTANCE HAS INTERNET ACCESS. TYPE THE ping commands TO CHECK THAT A WEBSITE HAS INTERNET ACCESS. TYPE THE COMMANDS BELOW.

ping ietf.org

ping [www.amazon.com](#)

ping [www.techdealerconsulting.com](#)

III) FROM YOUR NAT INSTANCE, CONNECT (SSH) TO YOUR PRIVATE INSTANCE IN YOUR PRIVATE SUBNET BY USING ITS PRIVATE IPV4 ADDRESS. TYPE THE COMMAND BELOW.

ssh ec2-user@private-server-private-ip-address

ssh [ec2-user@172.31.48.20](#)

IV) CONFIRM THAT YOUR PRIVATE INSTANCE CAN REACH THE INTERNET. TYPE THE ping commands TO CHECK THAT A WEBSITE HAS INTERNET ACCESS. TYPE THE COMMANDS BELOW.

ping ietf.org

ping [www.amazon.com](#)

ping www.techdealerconsulting.com

IF THE **ABOVE PINGS**, CONGRATULATIONS, YOU HAVE JUST SET UP A **NAT INSTANCE**. TYPE THE **COMMANDS BELOW** TO EXIT AND KILL THE **SSH AGENT**.

```
exit                                # exit the private instance
```

```
exit                                # exit the nat instance
```

```
eval $(ssh-agent -k)           # kill the ssh agent
```

8) TROUBLESHOOTING

If the **ping command** fails from **the private instance** in the **private subnet**, use the following steps to troubleshoot the issue.

- 1) Verify that the **security group** for your **NAT instance** allows inbound **ICMP** traffic from your **private subnet**. Otherwise, your **NAT instance** can't receive the **ping** command from your **private instance**.
- 2) Verify that you **disabled source/destination** checking for your **NAT instance**.
- 3) Verify that you configured your **route tables** correctly.

**TECH
DEALER**
consulting

DELIVERING TOMORROW'S TECH TODAY

